



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 5: Exclusión mutua con Test-and-Set

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 17 de septiembre de 2026

Índice general

Índice de figuras	III
1. Exclusión mutua con Test-and-Set	1
1.1. Código y ejecución en Java	3
2. Conclusiones	4

Índice de figuras

1.1. Captura del código Java y de la ejecución de Test-and-Set. . .	3
---	---

1. Exclusión mutua con Test-and-Set

Desarrollen un programa en Java que cree un cerrojo utilizando AtomicBoolean, inicialmente en false; cree tres hilos que compartan el mismo cerrojo y un contador; implemente una operación para solicitar acceso a la sección crítica utilizando `getAndSet(true)`; mantenga al hilo en espera activa mientras el cerrojo esté ocupado; incremente el contador y muestre el identificador y el valor del contador dentro de la sección crítica; libere el cerrojo cambiándolo a false; y repita el proceso cinco veces por hilo.

Entradas. Tres hilos identificados como $P0$, $P1$ y $P2$, un AtomicBoolean lockFlag compartido e inicialmente en false, un contador compartido inicializado en cero y cinco iteraciones por hilo. La prueba de inicio permite que los tres hilos comiencen a solicitar el cerrojo de manera concurrente.

Procesamiento. La sección de entrada ejecuta requestCS. En Java, la expresión lockFlag.getAndSet(true) obtiene el valor anterior y cambia el cerrojo a true como una sola operación atómica. Si el valor anterior es true, el hilo permanece en el ciclo while y realiza espera activa. Si el valor anterior es false, el hilo entra a la sección crítica, incrementa el contador, muestra su identificador y libera el cerrojo con getAndSet(false).

Salidas. Cada entrada muestra el hilo, la iteración y el valor del contador. Como existen tres hilos y cada uno realiza cinco entradas, el valor esperado es $3 \times 5 = 15$. El valor obtenido en las pruebas fue 15.

Estructuras de control utilizadas. Se emplean tres hilos, un ciclo for para repetir las cinco entradas, un ciclo while para la espera activa, métodos de solicitud y liberación del cerrojo y una sección crítica entre las secciones de entrada y salida. También se utilizan operaciones condicionales implícitas en el valor anterior que devuelve getAndSet.

Ejercicio 1. Exclusión mutua con Test-and-Set

Conceptos. Test-and-Set es una instrucción capaz de leer una variable y modificarla como una única operación atómica. Al convertir la comprobación y la modificación del cerrojo en una sola operación, solo un hilo puede obtener false cuando el cerrojo está libre. Los demás hilos obtienen true y permanecen en espera activa hasta que otro hilo libera el cerrojo. La espera activa consume tiempo de procesador, que es la limitación principal indicada en el módulo.

Secciones del programa.

- **Sección de entrada:** el método requestCS ejecuta repetidamente Test-and-Set.
- **Sección crítica:** el hilo incrementa el contador y muestra el resultado.
- **Sección de salida:** el método releaseCS cambia el cerrojo a false.
- **Sección restante:** la repetición de la prueba y la espera de los hilos al finalizar.

Prueba de escritorio. La siguiente secuencia representa el caso en que *P0* ya obtuvo el cerrojo y *P1* intenta entrar simultáneamente. Los valores se expresan como booleanos, de acuerdo con el enunciado.

Paso	Proceso	lockFlag antes	Operación	lockFlag después	Resultado
1	P0	false	TestAndSet(true) devuelve false	true	P0 entra a la sección crítica.
2	P1	true	TestAndSet(true) devuelve true	true	P1 no entra y comienza la espera activa.
3	P1	true	TestAndSet(true) devuelve true	true	P1 continúa esperando.
4	P0	true	TestAndSet(false)	false	P0 termina y libera el cerrojo.
5	P1	false	TestAndSet(true) devuelve false	true	P1 obtiene el cerrojo y entra.
6	P1	true	TestAndSet(false)	false	P1 termina y libera el cerrojo.

Evaluación. El algoritmo garantiza exclusión mutua porque la operación que obtiene el valor anterior y cambia el cerrojo es una operación atómica desde el punto de vista de los hilos que compiten. Cuando el cerrojo está libre, solo un hilo puede recibir false; los demás reciben true y no avanzan a la sección crítica. El contador final coincide con el esperado: 15.

Espera activa. La espera activa aparece en el ciclo while que contiene `getAndSet(true)`. En cada repetición el hilo vuelve a comprobar el cerrojo y cede brevemente el procesador, pero permanece listo para ejecutar. Esta solución evita la entrada simultánea, aunque consume CPU mientras existen hilos esperando.

1.1 Código y ejecución en Java

La figura reúne la captura del archivo Java y el resultado de su ejecución. Debe mostrar la creación de `AtomicBoolean`, la llamada a `getAndSet(true)`, la liberación con `false`, los tres hilos y el contador final igual a 15.

```

// Ejercicio07TestAndSet.java
05 Exclusion mutua con atomizidad : code > java > J Ejercicio07TestAndSet.java % Ejercicio07TestAndSet ? requestCSout
6
7 public class Ejercicio07TestAndSet {
8     private static final int HILOS = 3;
9     private static final int ITERACIONES = 5;
10    private static final AtomicBoolean lockFlag = new AtomicBoolean(initialValue: false);
11    private static final AtomicInteger contador = new AtomicInteger(initialValue: 0);
12    private static final CountDownLatch inicio = new CountDownLatch(count: 1);
13
14    // solicita acceso a la seccion critica mediante getAndSet
15    private static void requestCS(int id) {
16        while (!lockFlag.getAndSet(initialValue: true)) {
17            Thread.onSpinWait();
18        }
19    }
20
21    // libera al correa al salir de la seccion critica
22    private static void releaseCS(int id) {
23        lockFlag.getAndSet(initialValue: false);
24    }
25
26    // ejecuta las ordenes del hilo a la seccion critica
27    private static void procesa(int id) {
28        try {
29            inicio.await();
30            for (int iteracion = 1; iteracion <= ITERACIONES; iteracion++) {
31                requestCS(id);
32                int valor = contador.incrementAndGet();
33                System.out.println("hilo " + id
34                    + " entra a la seccion critica en la iteracion " + iteracion
35                    + ", contador = " + valor);
36                releaseCS(id);
37            }
38        } catch (InterruptedException e) {
39            Thread.currentThread().interrupt();
40        }
41    }
42
43    //main
44    public static void main(String[] args) throws InterruptedException {
45        Thread[] hilos = new Thread[HILOS];
46
47        System.out.println("Prueba de exclusion mutua con test-and-set");
48        for (int id = 0; id < HILOS; id++) {
49            int offset = id;
50            hilos[id] = new Thread(() -> procesa(idFinal, "hilo-" + id));
51            hilos[id].start();
52        }
53
54        inicio.countDown();
55
56        for (Thread hilo : hilos) {
57            hilo.join();
58        }
59
60        System.out.println("contador esperado: " + (HILOS * ITERACIONES));
61        System.out.println("contador obtenido: " + contador.get());
62        System.out.println("resultado: getAndSet garantiza exclusion mutua con un correa booleano");
63    }

```

```

Java Ejercicio07TestAndSet.java
prueba de exclusion mutua con test-and-set
hilo 2 entra a la seccion critica en la iteracion 1, contador = 1
hilo 1 entra a la seccion critica en la iteracion 1, contador = 2
hilo 0 entra a la seccion critica en la iteracion 1, contador = 3
hilo 2 entra a la seccion critica en la iteracion 2, contador = 4
hilo 1 entra a la seccion critica en la iteracion 2, contador = 5
hilo 0 entra a la seccion critica en la iteracion 2, contador = 6
hilo 2 entra a la seccion critica en la iteracion 3, contador = 7
hilo 1 entra a la seccion critica en la iteracion 3, contador = 8
hilo 0 entra a la seccion critica en la iteracion 3, contador = 9
hilo 2 entra a la seccion critica en la iteracion 4, contador = 10
hilo 1 entra a la seccion critica en la iteracion 4, contador = 11
hilo 0 entra a la seccion critica en la iteracion 4, contador = 12
hilo 2 entra a la seccion critica en la iteracion 5, contador = 13
hilo 1 entra a la seccion critica en la iteracion 5, contador = 14
hilo 0 entra a la seccion critica en la iteracion 5, contador = 15
contador esperado: 15
contador obtenido: 15
resultado: getAndSet garantiza exclusion mutua con un correa booleano

```

Figura 1.1: Captura del código Java y de la ejecución de Test-and-Set.

2. Conclusiones

La implementación en Java demuestra la función de Test-and-Set dentro de la exclusión mutua. El cerrojo booleano inicia libre, cada hilo solicita acceso con una única operación que obtiene el valor anterior y escribe true, y el hilo que logra entrar libera el cerrojo al salir. Por ello, tres hilos que realizan cinco entradas producen exactamente quince incrementos del contador.

La prueba de escritorio muestra la diferencia con una solución que primero comprueba y después modifica el cerrojo. Con Test-and-Set esas dos acciones se realizan como una operación atómica, así que dos hilos no pueden obtener false para la misma entrada. Su costo es la espera activa, que consume CPU mientras el cerrojo permanece ocupado.